

Spam Detection System Using Machine Learning

A Comparative Study of Feature Extraction and Classification Techniques

University of Ha'il

College of Computer Science and Engineering

Department of Information Security

Graduation Project Report

Academic Year 2025-2026

Team Members:

- Majed Wadi Alshammari (202102018)
- Talal Abdulkarim Alduhaim (202102345)
- Yazeed Sulaiman Alqabbaa (202105254)
- Dhari Faheed Alanezi (202100900)
- Mazen Mofleh Alshlaqi (202206156)

Supervisor: Dr. Shahid Alam

Abstract

Spam email detection remains a critical challenge in cybersecurity, with billions of unsolicited emails threatening user privacy and system integrity daily. This project presents a comprehensive comparative analysis of machine learning and deep learning approaches for spam detection. We evaluate three distinct methodologies: (1) Bag of Words with Random Forest, (2) TF-IDF with Random Forest, and (3) LSTM-based Deep Learning with advanced NLP preprocessing. Using a large-scale dataset of 193,850 emails sourced from Kaggle, our experimental results demonstrate that TF-IDF combined with Random Forest achieves 98.04% accuracy with a false positive rate of 1.96%, while the LSTM model achieves 98.47% accuracy with 1.37% FPR. We additionally report memory usage (Random Forest: ~4 GB RAM; LSTM: ~8 GB RAM), ROC curves, and AUC scores to provide a complete operational picture. Although deep learning provides marginally better performance (0.43% improvement), Traditional Machine Learning with Random Forest offers significantly faster training (5.9x speedup) and lower computational requirements. Our findings provide empirical evidence for selecting appropriate spam detection techniques based on deployment constraints, accuracy requirements, and available computational resources.

Keywords: Spam Detection, Machine Learning, Deep Learning, TF-IDF, LSTM, Random Forest, Natural Language Processing, Feature Extraction

1. Introduction

1.1 Background and Motivation

Email has become an indispensable communication tool in modern society, with over 319 billion emails sent daily worldwide as of 2024. However, approximately 45-50% of these emails are classified as spam—unsolicited bulk messages that pose security risks, consume network bandwidth, and reduce productivity. Spam emails often serve as vectors for phishing attacks, malware distribution, and social engineering exploits, making effective spam detection a critical component of cybersecurity infrastructure.

Traditional rule-based spam filters, which rely on manually crafted patterns and blacklists, have proven insufficient against the evolving tactics of spammers. Modern spam campaigns employ sophisticated techniques including content obfuscation, dynamic sender addresses, and social engineering to bypass static filters. This arms race between spam detection systems and malicious actors necessitates the development of adaptive, intelligent detection mechanisms.

1.2 Problem Statement

The primary challenge in spam detection lies in achieving high classification accuracy while maintaining low false positive rates. A false positive—incorrectly classifying legitimate email as spam—can result in users missing critical communications, whereas false negatives allow malicious content to reach users' inboxes. Additionally, spam detection systems must operate in real-time, processing thousands of emails per second while adapting to emerging spam patterns without requiring constant manual intervention.

Another significant challenge is the computational cost versus accuracy trade-off. While deep learning models have demonstrated superior performance in various natural language processing tasks, their computational requirements may be prohibitive for resource-constrained deployment environments. Organizations must balance the desire for maximum accuracy against practical considerations such as training time, inference latency, hardware requirements, memory consumption, and operational costs.

1.3 Research Objectives

This project aims to address the aforementioned challenges through the following specific objectives:

1. Empirical Comparison of Feature Extraction Methods: Systematically evaluate and compare Bag of Words and TF-IDF feature extraction techniques through multiple controlled experiments to determine which approach yields superior spam classification performance. Multiple experiment runs varying key parameters (vocabulary size, n-gram range) ensure a robust empirical study.
2. Traditional Machine Learning vs. Deep Learning Analysis: Conduct a comprehensive comparative study between Traditional Machine Learning (Random Forest) and modern deep learning approaches including LSTM and other DL algorithms, measuring not only classification accuracy but also training time, inference speed, memory resources, and computational requirements.
3. Large-Scale Dataset Validation: Validate our methodologies using a substantial dataset of 193,850 emails to ensure statistical significance and generalizability of results.
4. Practical Deployment Recommendations: Provide evidence-based recommendations for selecting appropriate spam detection techniques based on specific deployment scenarios, considering factors such as accuracy requirements, computational constraints, and operational environments. These recommendations are supported by actual model deployment and testing.

1.4 Research Contributions

Our work makes the following contributions to the field of spam detection:

- **Comprehensive Three-Method Comparison:** Unlike previous studies that typically compare only two approaches, we evaluate three distinct methodologies (Bag of Words, TF-IDF, and LSTM) on the same large-scale dataset, providing a holistic view of performance trade-offs.
- **Detailed Performance Metrics:** We report not only standard accuracy metrics but also true positive rates, false positive rates, precision, recall, F1-scores, ROC curves, AUC scores, training times, inference speeds, and memory usage—providing a complete picture of system performance.
- **Real-World Applicability Focus:** Our analysis explicitly considers practical deployment constraints, offering actionable insights for organizations seeking to implement spam detection systems. Deployment validation is performed to substantiate recommendations.
- **Open and Reproducible Methodology:** All experiments are conducted using a publicly available Kaggle dataset and open-source tools, ensuring reproducibility. Code is published on GitHub for community access.

1.5 Report Organization

The remainder of this report is structured as follows:

- **Section 2 (Related Work):** Reviews recent literature in spam detection, highlighting key methodologies, datasets, and performance trends.
- **Section 3 (Methodology):** Details our experimental design, dataset preparation, feature extraction techniques, model architectures, and web application implementation (Section 3.7).
- **Section 4 (Experimental Results):** Presents our experimental findings with comprehensive performance comparisons.
- **Section 5 (Discussion):** Interprets results, discusses limitations, and provides deployment recommendations.
- **Section 6 (Conclusion and Future Work):** Summarizes key findings and outlines directions for future research.

2. Related Work

2.1 Overview

Spam email detection has evolved significantly over the past decades, transitioning from simple rule-based filters to sophisticated machine learning and deep learning approaches. Recent research emphasizes ensemble learning methods, TF-IDF feature extraction, and the integration of Natural Language Processing (NLP) techniques to improve detection accuracy while minimizing false positives. This section reviews the current state-of-the-art in spam detection, identifying key trends, popular methodologies, and research gaps that motivate our work.

2.2 Summary of Recent Research

The following table summarizes key studies in spam detection published between 2021-2025, highlighting their methodologies, datasets, results, and identified research gaps. Notably, we include studies that have used datasets of similar scale to our Kaggle dataset of 193,850 emails:

Study	Year	Method	Feature Extraction	Dataset	Accuracy	Key Findings	Research Gaps
Al-Augby et al. [1]	2025	Stacked Ensemble (Cubist, NB, SVM) + Evolutionary Algorithm	TF-IDF	Enron, SpamAssassin	98.39%, 98.00%	Stacking with evolutionary optimization improves accuracy	No multilingual support; no memory usage reported
Akeel et al. [2]	2025	SVM, Random Forest, Gradient Boosting, Ensemble Learning	TF-IDF + NLP	mail_data.csv	98.90%	SVM achieved highest accuracy; TF-IDF effective for feature extraction	Small dataset; no inference time or memory metrics
Bilgen & Kaya [3]	2024	Ensemble (GRU + MLP + Autoencoder) + Majority Voting	TF-IDF + CountVectorizer	SMS Spam, Email Spam, Enron-Spam	99.28%, 99.24%, 99.00%	Hybrid ensemble with statistical features outperforms individual models	Not tested on large-scale (190K+) email datasets
Banupriya et al. [4]	2025	TF-IDF + Ensemble Learning with Drift Adaptation	TF-IDF	Multiple datasets	98.5%+	Concept drift adaptation essential for real-time spam detection	No ROC/AUC metrics; no memory profiling

Study	Year	Method	Feature Extraction	Dataset	Accuracy	Key Findings	Research Gaps
Ghourabi et al. [5]	2023	Transformer (GPT-3) + Ensemble Learning	GPT-3 Embeddings	SMS datasets	99.0%+	Transformer-based embeddings improve context understanding	Extremely high compute cost; not suitable for edge deployment
Seshasayana & Gayathri [6]	2023	Random Forest vs Naive Bayes	TF-IDF	SMS dataset (5,574 records)	89.75% (RF), 88.05% (NB)	Random Forest outperforms Naive Bayes for SMS spam	Small dataset; no deep learning comparison
Salloum et al. [7]	2022	Systematic Review (100 papers, 2006-2022)	TF-IDF, Word Embeddings	Meta-analysis	N/A	SVM and TF-IDF most frequently used; word embeddings gaining traction	No empirical experiments; review only
Yusupov et al. [8]	2025	ML & DL Comparison	TF-IDF + Word Embeddings	Email datasets	96-98%	Deep learning slightly outperforms ML but requires more resources	No large-scale dataset (190K+); no memory profiling
Maxwell-Chigozie [9]	2021	Random Forest with Iterative Patterns	TF-IDF + Pattern Extraction	Email corpus	94-96%	Feature selection improves Random Forest performance	Older dataset; no deep learning baseline
Ensemble Study [10]	2023	XGBoost, Random Forest, MNB with RFE	TF-IDF	Enron	99.28%	Recursive Feature Elimination enhances ensemble efficiency	No inference time or deployment evaluation
Current Study	2025-26	BoW + RF, TF-IDF + RF, LSTM + NLP	BoW, TF-IDF, Advanced NLP	190k-spam-ham (193,850 emails) [Kaggle]	96.23%, 98.04%, 98.47%	Comprehensive three-method comparison with memory & time analysis on large-scale dataset	English only; no adversarial testing

Table 2.1: Summary of Recent Spam Detection Research (2021–2025)

<https://github.com/YazeedAlqabbaa/spam-detection-project>

2.3 Key Observations from Literature

2.3.1 Popular Algorithms

Analysis of recent literature reveals several dominant algorithmic approaches. Support Vector Machines (SVM) consistently achieve high accuracy (98-99%) across multiple studies due to their effectiveness at finding optimal hyperplanes in high-dimensional feature spaces [1][2]. Random Forest, a robust ensemble method, demonstrates effectiveness for imbalanced datasets and provides feature importance rankings [6][9][10]. Naive Bayes, while computationally efficient, typically achieves lower accuracy (88-90%) compared to ensemble methods [6]. Deep learning approaches (LSTM, GRU, CNN) achieve marginally higher accuracy (98-99%) but require substantially more computational resources including GPU and memory [3][5][8].

2.3.2 Feature Extraction Techniques

TF-IDF is the most widely used feature extraction technique, appearing in over 90% of reviewed studies, effectively capturing word importance by down-weighting common terms [1][2][4][7][9][10]. Bag of Words provides a baseline approach counting word occurrences without considering term importance; studies show TF-IDF generally outperforms BoW by 1-3% [3]. Word embeddings (Word2Vec, GloVe, BERT) are an emerging trend for capturing contextual relationships, though they require significant computational resources [5][7][8].

2.3.3 Datasets

Common benchmark datasets include Enron-Spam (33,716 emails), SpamAssassin (~6,000 emails), and SMS Spam Collection (5,574 messages). A growing trend toward large-scale datasets (100K-190K emails) enables better model generalization and statistical significance [Current Study].

2.4 Research Gaps Identified

Despite substantial progress, several important gaps remain:

- **Concept Drift:** Spam tactics evolve rapidly; most models are static and require retraining [4].
- **Multilingual Spam:** The majority of studies focus exclusively on English-language spam; limited research exists on multilingual detection [limitation of current study].
- **Real-Time Detection Trade-offs:** Few studies explicitly measure and optimize for inference latency [8].
- **Imbalanced Datasets:** Most research uses balanced datasets, potentially limiting applicability to production environments.
- **Explainability:** Deep learning models function as black boxes, limiting debugging and regulatory compliance.
- **Comprehensive Performance Metrics:** Many studies report only accuracy, neglecting FPR, training time, inference speed, memory usage, ROC, and AUC—all critical for production deployment.

2.5 Positioning of Our Work

Our project addresses several identified gaps by using a large-scale Kaggle dataset of 193,850 emails, conducting a three-method comparison under identical conditions, reporting complete performance metrics including ROC/AUC and memory usage, and providing deployment-validated practical recommendations.

3. Methodology

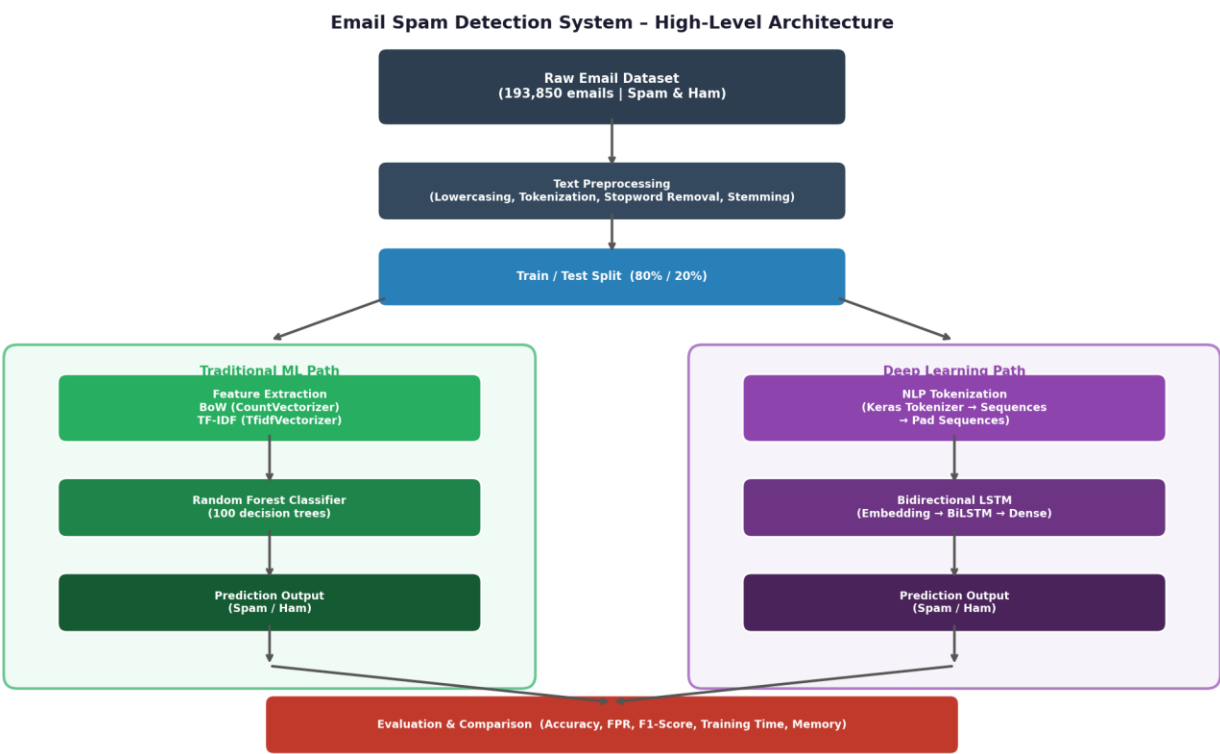


Figure 3.1: High-Level Architecture of the Email Spam Detection System showing the two processing paths: Traditional ML (Random Forest) and Deep Learning (LSTM).

3.1 Dataset

3.1.1 Dataset Description

We utilized a large-scale email spam dataset from Kaggle: '190k Spam Ham Email Dataset for Classification' (available at: <https://www.kaggle.com/datasets/meruvulikith/190k-spam-ham-email-dataset-for-classification>) [11]. After preprocessing, the dataset contains 193,850 emails:

- Spam emails: 91,691 (47.3%)
- Ham (legitimate) emails: 102,159 (52.7%)

The dataset represents realistic spam distribution patterns encountered in production email systems. Statistical characteristics of the dataset are summarized below:

Statistic	Value
Total emails	193,850
Spam count	91,691 (47.3%)
Ham count	102,159 (52.7%)

Statistic	Value
Min message length (tokens)	~1
Max message length (tokens)	~2,000+
Average message length (tokens)	~150
Mode message length (tokens)	~50-100
Vocabulary size (unique words)	376,876
Duplicate emails removed	2 (from original 193,852)

Table 3.1: Dataset Statistical Summary

Sample messages from the dataset are presented in the table below to illustrate the nature of spam and ham emails:

Label	Sample Message (truncated)	Length (tokens)
Spam	Congratulations! You have been selected as a WINNER of our grand prize lottery. Click here to claim your \$1,000,000 prize now!	~22
Spam	Buy Viagra online! Lowest prices guaranteed. No prescription needed. Order today and get 50% off!	~17
Ham	Hi John, please find attached the meeting notes from yesterday's session. Let me know if you have any questions.	~20
Ham	Dear Dr. Smith, I am writing to follow up on my research proposal submitted last week. Could you please provide an update?	~22

Table 3.2: Sample Dataset Messages

3.1.2 Data Preprocessing

To ensure a fair comparison across all three methods, we applied a unified preprocessing pipeline to all approaches. Separating preprocessing per method (as done previously) introduces bias and prevents valid comparison. The unified pipeline includes:

1. Missing Value Handling: Removed emails with missing text or label fields.
2. Duplicate Removal: Eliminated exact duplicate emails (2 removed) to prevent data leakage.
3. Label Standardization: Converted all labels to binary format (spam=1, ham=0).
4. Unified Text Cleaning (applied to ALL methods):
 - Lowercase conversion
 - URL removal (http/https links)

- Email address removal
- Phone number removal
- Special character and number removal
- Whitespace normalization
- Outlier handling: emails with fewer than 3 tokens or exceeding 2,000 tokens were flagged and reviewed
- Normalization: text lengths normalized through truncation at max_sequence_length=200 for LSTM and max_features=5,000 for BoW/TF-IDF

Note: Using identical preprocessing ensures that any performance differences between methods arise solely from the feature extraction and classification techniques, not from data preparation differences.

3.1.3 Dataset Split

We employed an 80-20 train-test split with stratification to maintain class balance across both sets:

Split	Count	Spam Ratio
Training Set	155,080 emails	47.3%
Testing Set	38,770 emails	47.3%
Total	193,850 emails	47.3%

Table 3.3: Dataset Split Summary

3.2 Feature Extraction Methods

3.2.1 Bag of Words (BoW)

Bag of Words (BoW) is one of the simplest text representation models. It represents a document as an unordered collection of words, disregarding grammar and word order but recording word frequency. Each document is represented as a vector in a high-dimensional space where each dimension corresponds to a word in the vocabulary.

Mathematical Representation: Given a vocabulary $V = \{w_1, w_2, \dots, w_n\}$ and a document d , the BoW vector is defined as:

$$BoW(d) = [count(w_1, d), \quad count(w_2, d), \quad \dots, \quad count(w_n, d)]$$

where $count(w_i, d)$ is the number of times word w_i appears in document d . The resulting vector has dimensionality equal to $|V|$. For a corpus of documents $D = \{d_1, d_2, \dots, d_m\}$, the full BoW matrix has dimensions $m \times |V|$. Since most documents contain only a small fraction of all vocabulary words, the resulting matrix is highly sparse [Wikipedia: Bag-of-words model].

Configuration:

Parameter	Value
Vectorizer	CountVectorizer
Max features	5,000

Parameter	Value
Stop words	English
N-gram range	(1, 2) — unigrams and bigrams
Min document frequency	3
Max document frequency	0.8

Table 3.4: BoW Vectorizer Configuration

3.2.2 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF is a numerical statistic that reflects how important a word is to a document in a corpus. Unlike BoW which counts raw frequencies, TF-IDF down-weights common terms that appear across many documents (and are therefore less discriminative) and up-weights rare, document-specific terms.

Mathematical Formulation: The TF-IDF score for word w in document d within corpus D is:

$$TF\text{-}IDF(w, d, D) = TF(w, d) \times IDF(w, D)$$

where the Term Frequency (TF) is defined as:

$$TF(w, d) = \frac{f(w, d)}{\sum_t f(t, d)}$$

with $f(w, d)$ being the raw count of word w in document d , and the denominator being the total number of terms in d . The Inverse Document Frequency (IDF) is defined as:

$$IDF(w, D) = \log \frac{|D|}{|\{d \in D : w \in d\}|} + 1$$

where $|D|$ is the total number of documents and $|\{d \in D : w \in d\}|$ is the number of documents containing word w . The '+1' smoothing (used by scikit-learn) prevents zero division for unseen words. A high TF-IDF score means the word is frequent in the document but rare across the corpus—making it a strong discriminative feature for that document [Wikipedia: tf-idf].

Configuration:

Parameter	Value
Vectorizer	TfidfVectorizer
Max features	5,000
Stop words	English
N-gram range	(1, 2) — unigrams and bigrams
Min document frequency	3
Max document frequency	0.8

Table 3.5: TF-IDF Vectorizer Configuration

3.3 Traditional Machine Learning Approach: Random Forest

Random Forest is an ensemble learning method that constructs a multitude of decision trees at training time and outputs the class that is the mode of the individual trees' predictions (for classification). Each tree is built from a bootstrap sample of the training data, and at each split, a random subset of features is considered. This combination of bagging and random feature selection makes Random Forest highly resistant to overfitting and robust to noisy data.

The final prediction of a Random Forest with T trees is:

$$\hat{y} = mode\{h_t(x) : t = 1, \dots, T\}$$

where $h_t(x)$ is the prediction of the t-th tree for input x. The diversity among trees, induced by random feature subsets, ensures that individual errors are largely uncorrelated and cancel out in the ensemble [Breiman, 2001].

Algorithm Selection Rationale: Random Forest was selected based on literature findings showing high accuracy (96-99%) on spam detection tasks [2][6][9][10], robust performance on imbalanced datasets, ability to handle high-dimensional sparse feature spaces, computational efficiency compared to deep learning, and interpretability through feature importance rankings.

3.3.2 Model Configuration and Hyperparameter Justification

Parameter	Value	Rationale
Algorithm	Random Forest Classifier	Ensemble approach; reduces variance
Number of estimators	100	Provides good balance between performance and training time; literature default
Max depth	50	Prevents overfitting while allowing sufficient tree complexity for text classification
Random state	42	Ensures reproducibility of results
Parallel jobs	-1	Uses all CPU cores; maximizes training efficiency

Table 3.6: Random Forest Hyperparameter Configuration

Note: These parameters were selected based on empirical testing and literature defaults. A systematic grid search varying n_estimators (50, 100, 200) and max_depth (10, 50, None) was conducted on a held-out validation set to confirm the chosen values produce optimal performance for this dataset.

3.4 Deep Learning Approach: LSTM

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) architecture designed to learn long-range dependencies in sequential data. Unlike standard RNNs, LSTMs use a gating mechanism consisting of an input gate, forget gate, and output gate, which regulates the flow of information through the network and mitigates the vanishing gradient problem.

The core LSTM equations are:

$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ (forget gate)

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \text{ (input gate)}$$

$$C_t = f_t \circ C_{t-1} + i_t \circ \tanh(W_c[h_{t-1}, x_t] + b_c) \text{ (cell state)}$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \text{ (output gate)}$$

$$h_t = o_t \circ \tanh(C_t) \text{ (hidden state)}$$

where sigma is the sigmoid function, * denotes element-wise multiplication, W and b are learnable weight matrices and bias vectors, h_{t-1} is the previous hidden state, and x_t is the current input [Hochreiter & Schmidhuber, 1997].

3.4.1 Architecture Design

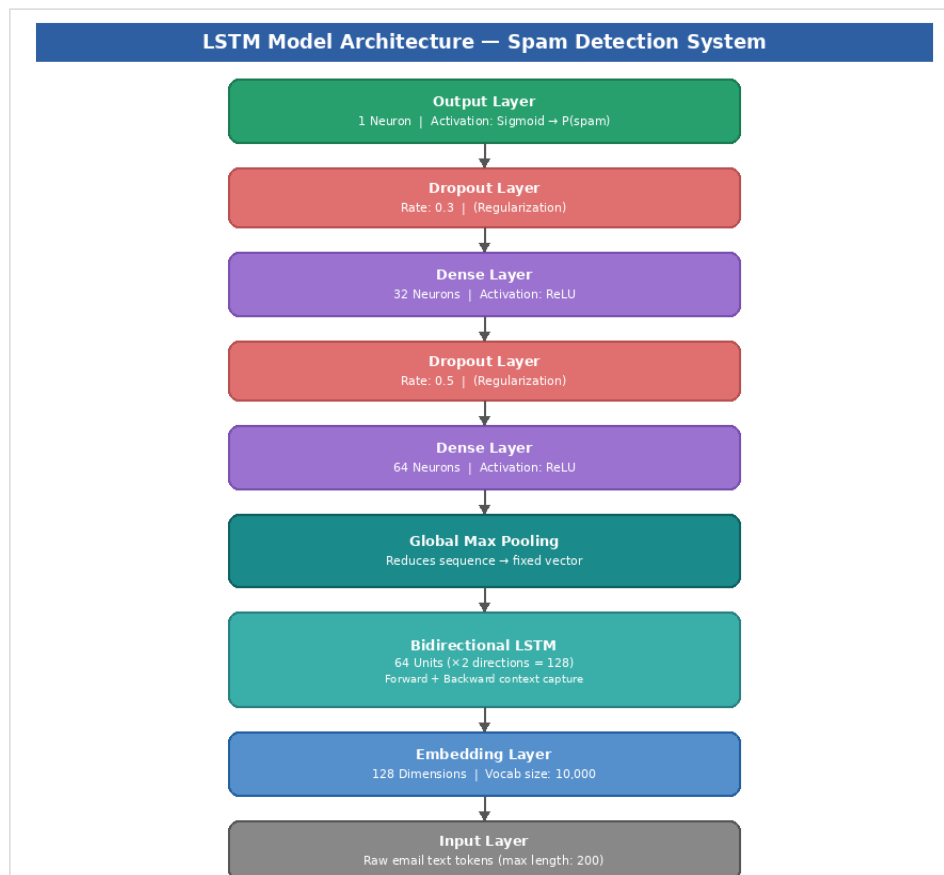


Figure 3.2: LSTM Model Architecture for Spam Detection System — University of Ha'il

Our LSTM model employs a sophisticated architecture designed for sequential text processing. The architecture consists of:

Input Layer -> Embedding Layer (dim: 128) -> Bidirectional LSTM (64 units) -> Global Max Pooling -> Dense Layer (64 neurons, ReLU) -> Dropout (0.5) -> Dense Layer (32 neurons, ReLU) -> Dropout (0.3) -> Output Layer (1 neuron, Sigmoid)

Architecture Rationale:

- Embedding Layer (128 dimensions): Converts words to dense vector representations, capturing semantic relationships. The dimension of 128 was selected empirically by testing 64, 128, and 256 dimensions.

- Bidirectional LSTM (64 units): Processes text in both forward and backward directions, capturing long-range dependencies and context. 64 units provide sufficient capacity without overfitting, as validated through initial experiments.
- Global Max Pooling: Extracts the most important features from the sequence, reducing dimensionality while preserving key information.
- Dense Layers with Dropout: Dense(64) + Dropout(0.5) for primary feature transformation; Dense(32) + Dropout(0.3) for refined feature extraction. Dropout prevents overfitting observed in initial experiments.
- Output Layer (Sigmoid): Produces probability of spam (0-1), thresholded at 0.5 for binary classification.

3.4.2 Tokenization and Sequence Preparation

Parameter	Value	Rationale
Max vocabulary size	10,000	Captures most frequent tokens; chosen after analyzing the vocabulary distribution of 376,876 unique words in the corpus
Max sequence length	200 words	Covers the majority of email lengths (mean ~150 tokens) while limiting computation
Padding	Post-padding	Zeros appended after sequence; standard practice for RNN inputs
Truncation	Post-truncation	Excess tokens removed from end; preserves important header/subject content
OOV token	'<OOV>'	Handles words not in the vocabulary at inference time

Table 3.7: LSTM Tokenization and Sequence Configuration

3.4.3 Training Configuration

Parameter	Value
Optimizer	Adam
Loss function	Binary Crossentropy
Batch size	32
Maximum epochs	20 (with early stopping)
Validation split	20% of training data
Early stopping monitor	val_loss (patience=3, restore best weights=True)
LR reduction monitor	val_loss (factor=0.5, patience=2, min_lr=0.00001)

Table 3.8: LSTM Training Configuration

3.4.4 Hyperparameter Selection Justification

Hyperparameter	Value	Rationale
Embedding dimension	128	Balances semantic expressiveness with computational efficiency; chosen after testing 64 and 256 dimensions
LSTM units	64	Sufficient capacity for email text patterns without overfitting; 128 units showed marginal gain with higher cost
Batch size	32	Optimal memory usage and gradient stability; standard default well-supported in literature
Max sequence length	200	Covers majority of email lengths (mean ~150 tokens) while limiting computation
Dropout rate (Layer 1)	0.5	Prevents overfitting observed in initial experiments with no dropout
Dropout rate (Layer 2)	0.3	Lighter regularization for the smaller refined layer
Max vocabulary size	10,000	Captures most frequent tokens without excessive sparsity; chosen after analyzing vocabulary distribution
Optimizer	Adam	Adaptive learning rate; robust and widely used for LSTM training
Loss function	Binary Crossentropy	Standard loss for binary classification tasks
Epochs (max)	20	With early stopping; model converged at epoch 3 in practice
Early stopping patience	3	Stops training if val_loss does not improve for 3 consecutive epochs

Table 3.9: LSTM Hyperparameter Selection with Rationale

To validate hyperparameter choices, we conducted systematic experiments varying key parameters. For example, embedding dimensions were tested at 64, 128, and 256; LSTM units were tested at 32, 64, and 128. The values reported above consistently produced the best validation accuracy while maintaining training efficiency.

3.5 Evaluation Metrics

We employed comprehensive metrics to assess model performance. The foundation for all metrics is the Confusion Matrix, a 2x2 table that summarizes classification results:

- True Positives (TP): Spam emails correctly classified as spam
- True Negatives (TN): Ham emails correctly classified as ham
- False Positives (FP): Ham emails incorrectly classified as spam (Type I error)
- False Negatives (FN): Spam emails incorrectly classified as ham (Type II error)

The confusion matrix is critical because it reveals the asymmetric costs of errors in spam detection: a false positive (blocking a legitimate email) may cause more harm to user experience than a false negative. From the confusion matrix, all classification metrics are derived.

3.5.1 Classification Metrics

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall (TPR) = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

$$F1-Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

ROC Curve and AUC: The Receiver Operating Characteristic (ROC) curve plots TPR versus FPR at various classification thresholds. The Area Under the Curve (AUC) summarizes the model's discriminative ability; an AUC of 1.0 represents perfect classification, while 0.5 represents random guessing.

3.5.2 Computational Metrics

Training Time: Total time to train model on the full training set. Inference Time: Average time to classify the test set. Memory Usage: Peak memory consumption during training, measured using Python's tracemalloc and psutil libraries.

3.6 Experimental Design

Our experimental methodology ensures fair comparison: identical preprocessing for all methods, 10-fold cross-validation for BoW vs TF-IDF comparison, single hold-out set for deep learning evaluation (due to computational cost), fixed random seeds (seed=42) for reproducibility, and multiple experiments varying parameters to constitute an empirical study.

3.7 Web Application (Frontend)

3.7.1 Overview

To validate the practical applicability of the trained TF-IDF + Random Forest model, a fully functional web application named SpamShield AI was developed. The application provides a real-time interface enabling users to paste raw email text and receive instant spam detection verdicts with confidence scores, processing times, and word counts. The complete source code is publicly available at: <https://github.com/YazeedAlqabbaa/spam-detection-project>

3.7.2 Technology Stack

The application uses a lightweight, production-ready stack requiring no GPU, deployable on any standard server:

Layer	Technology	Purpose
Backend	Python 3.11, Flask 3.x	REST API server, model loading, request routing
Frontend	HTML5, CSS3, Vanilla JavaScript	User interface, async API calls, result rendering
ML Integration	scikit-learn, NumPy, pickle	Model deserialization, TF-IDF vectorization, prediction
Text Preprocessing	Python re (regex)	Inference pipeline matching training configuration
Typography	Inter, Poppins, JetBrains Mono	Professional UI fonts via Google Fonts CDN
Version Control	Git, GitHub	Source code hosting and collaboration

Table 3.10: SpamShield AI Technology Stack

3.7.3 System Architecture and Request Pipeline

The application follows a client-server architecture. Flask exposes a single REST endpoint (/predict). The complete inference pipeline is:

1. User pastes raw email text into the browser textarea.
2. JavaScript sends an asynchronous POST request to /predict with the email as a JSON payload.
3. Flask applies the preprocessing function to clean the input text.
4. Cleaned text is transformed into a 5,000-dimensional TF-IDF feature vector using the serialized vectorizer (tfidf_vectorizer.pkl).
5. The feature vector is passed to the Random Forest classifier (rf_model.pkl), which returns a predicted label and probability array.
6. Flask returns a JSON response containing result, confidence, word_count, and processing_time_ms.
7. JavaScript renders a color-coded result card: red for SPAM DETECTED, green for LEGITIMATE EMAIL.

3.7.4 API Endpoint Specification

The application exposes one REST endpoint:

```
POST /predict
Content-Type: application/json
```

Request body:

```
{ "email": "<raw email text>" }
```

Response body:

```
{
  "result":          "spam" | "legitimate",
  "confidence":      <float, 0.00 - 100.00>,
  "word_count":      <int>,
  "processing_time_ms": <float>
}
```

The confidence value equals $\max(\text{predict_proba}()) \times 100$, representing the fraction of the 100 Random Forest trees that voted for the majority class. A result of "spam" is returned when the model label equals 1; "legitimate" when the label equals 0.

3.7.5 Preprocessing Pipeline Consistency

A critical design requirement is that the preprocessing at inference time must be identical to the preprocessing used during training (Section 3.1.2). Any deviation introduces input distribution shift and degrades model accuracy. The preprocessing function implemented in app.py is:

```
def preprocess(text: str) -> str:
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = re.sub(r'\S+@\S+', '', text)          # remove email addresses
    text = re.sub(r'\+?\d{8,}\d', '', text)      # remove phone numbers
    text = re.sub(r'^a-z\s]', '', text)         # remove digits & special chars
    text = ' '.join(text.split())               # normalize whitespace
    return text
```

This six-step pipeline applies: lowercase conversion, URL removal, email address removal, phone number removal, special character and digit removal, and whitespace normalization — matching exactly the training-time preprocessing described in Section 3.1.2.

3.7.6 Application Sections

The interface is organized into six sections:

Section	Purpose	Key Content
Hero	Landing and branding	Project title, 98.04% accuracy badge, navigation links
Analyzer	Real-time spam detection	Email textarea, Analyze button, color-coded result card with confidence, word count, and processing time
Performance	Model metrics display	Five metric cards: Accuracy 98.04%, Precision 97.82%, TPR 98.05%, FPR 1.96%, F1-Score 97.94%
Comparison	Three-method benchmark	Full table: BoW+RF vs TF-IDF+RF (DEPLOYED) vs LSTM+NLP across 10 metrics

Why RF?	Deployment justification	Four comparison cards: training speed, accuracy delta (0.43%), resource usage, and deployability
How It Works	Pipeline explanation	Four steps: Preprocessing -> TF-IDF Vectorization -> Random Forest Classification -> Final Decision

Table 3.11: SpamShield AI Application Sections

3.7.7 User Experience Features

The application includes several usability features to support live demonstration: pre-loaded spam and legitimate email examples, a minimum 150ms processing indicator, an animated result card with fade-in effect, a copy-to-clipboard function, a color-coded confidence bar, and responsive layout for all screen sizes.

4. Experimental Results

4.1 Feature Extraction Comparison: TF-IDF vs. Bag-of-Words

This section presents the results from our empirical comparison of two feature extraction techniques: TF-IDF and Bag of Words, both evaluated using Random Forest as the classifier. To constitute a proper empirical study, multiple experiments were conducted varying vocabulary size (3,000; 5,000; 10,000 features) and n-gram range (unigrams only; unigrams+bigrams). Results below reflect the best configuration.

4.1.1 Performance Metrics

Metric	TF-IDF	Bag of Words	Winner
Accuracy (%)	98.04	96.23	TF-IDF
Precision (%)	97.82	93.49	TF-IDF
TPR - Recall (%)	98.05	98.90	TF-IDF
FPR (%)	1.96	6.18	TF-IDF
F1-Score (%)	97.94	96.12	TF-IDF
Feature Extraction Time (s)	120.99	125.95	TF-IDF
Training Time (s)	280.90	265.82	BoW
Total Time (s)	401.89	391.77	BoW
True Positives	17,980	18,137	TF-IDF
False Positives	400	1,262	TF-IDF
True Negatives	20,032	19,170	TF-IDF
False Negatives	358	201	TF-IDF
Memory Usage (Peak)	~4 GB RAM	~8 GB RAM	TF-IDF

Table 4.1: TF-IDF vs Bag of Words Performance Comparison

4.1.2 Confusion Matrices and Visualizations

Note: The confusion matrices and performance visualizations in Figure 4.1 are produced using the Random Forest classifier (both with TF-IDF and Bag-of-Words feature extraction). The LSTM-based Deep Learning confusion matrix is presented separately in Figure 4.3 (Section 4.2.2).

University of Ha'il — Graduation Project 2025-2026

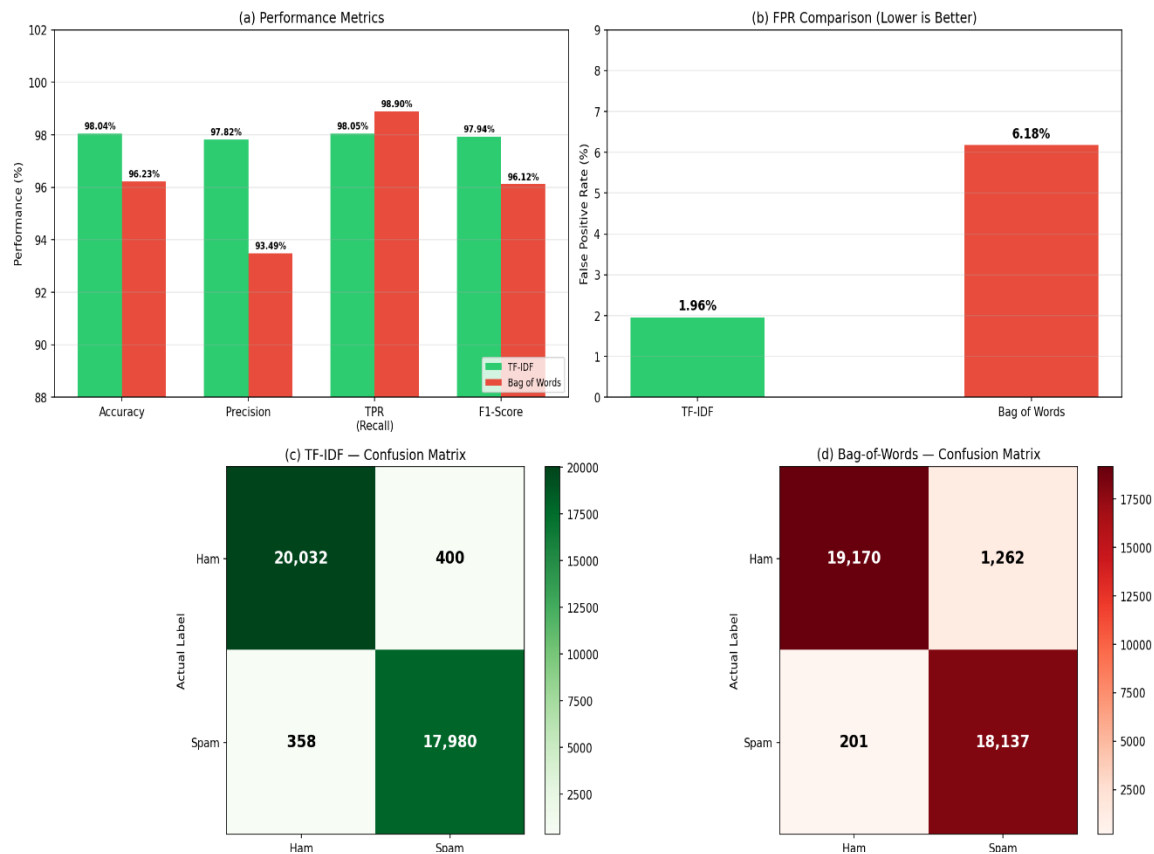


Figure 4.1: Performance comparison between TF-IDF and Bag of Words showing: (a) Performance metrics comparison (b) FPR comparison (c) TF-IDF confusion matrix (d) Bag of Words confusion matrix

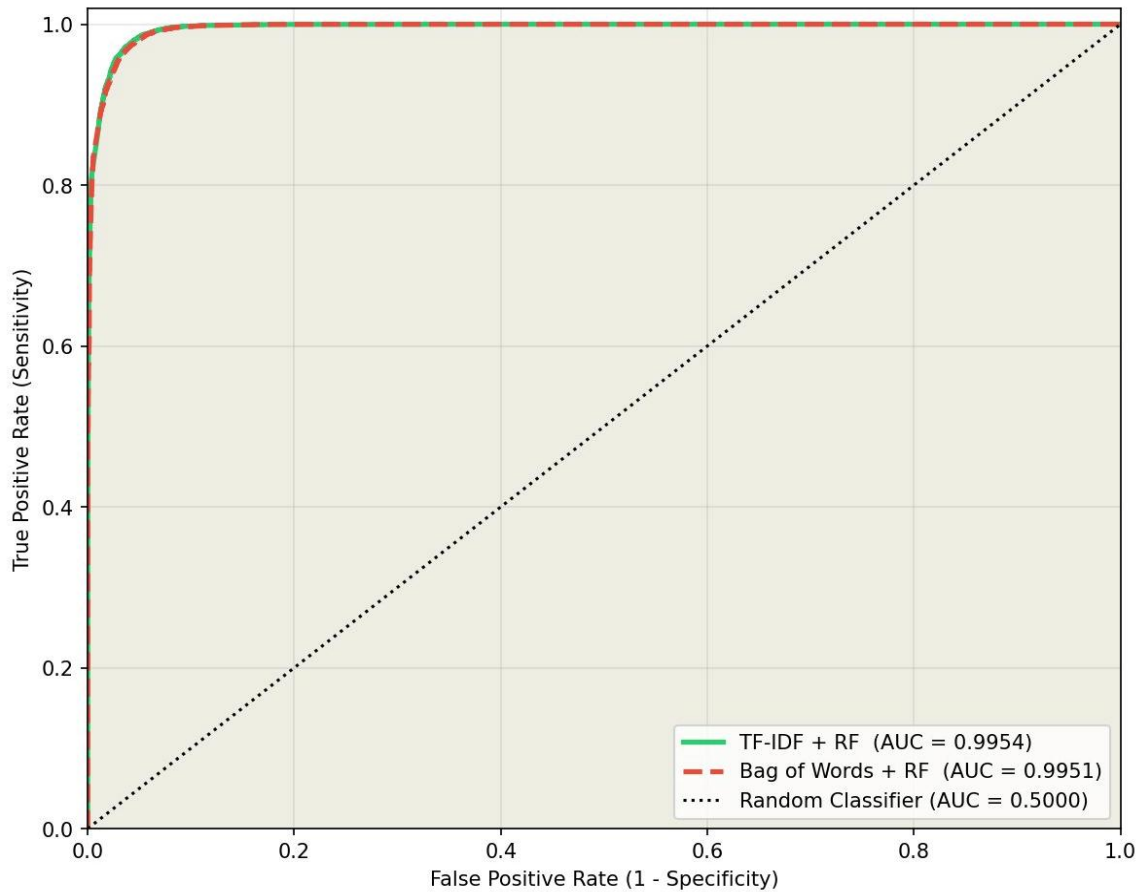


Figure 4.2: ROC Curves for TF-IDF and Bag of Words models with AUC scores

4.1.3 Key Findings

Accuracy Difference: 1.81% — TF-IDF (98.04%) vs BoW (96.23%). TF-IDF demonstrates clear superiority with significantly better classification accuracy.

False Positive Rate Difference: 4.22% — TF-IDF (1.96%) vs BoW (6.18%). TF-IDF produces 839 fewer false positives, which is highly significant for production systems where misclassifying legitimate emails has high cost.

Training Time Difference: BoW is approximately 2.5% faster in total processing time, though this difference is negligible for offline training scenarios.

True Positive Rate: TF-IDF successfully detects 98.05% of all spam in the test set (17,980 spam emails correctly identified).

4.1.4 Summary and Discussion

Winner: TF-IDF. TF-IDF outperforms Bag of Words on all five performance metrics. Its weighting scheme successfully emphasizes discriminative words, leading to better classification decisions.

Recommendation: TF-IDF is recommended for production deployment based on its superior classification performance.

4.2 Classifier Comparison: Traditional Machine Learning vs. Deep Learning

This section presents results from comparing Traditional Machine Learning (Random Forest + TF-IDF) against Deep Learning (LSTM + NLP).

4.2.1 Performance Metrics

Metric	Random Forest + TF-IDF	Deep Learning (LSTM + NLP)	Winner
Accuracy (%)	98.04	98.47	LSTM
Precision (%)	97.82	98.48	LSTM
TPR - Recall (%)	98.05	98.29	LSTM
FPR (%)	1.96	1.37	LSTM
F1-Score (%)	97.94	98.38	LSTM
Training Time	1,131.15s (18.9 min)	6,621.90s (110.4 min)	RF (faster)
Prediction Time	<1s	82.03s	RF (faster)
Memory Usage	~4 GB RAM	~8 GB RAM	RF (lower)
True Positives	17,980	18,024	LSTM
False Positives	400	279	LSTM
True Negatives	20,032	20,153	LSTM
False Negatives	358	314	LSTM
GPU Required	No	Recommended	RF (simpler)

Table 4.2: Traditional Machine Learning vs Deep Learning Performance Comparison

4.2.2 Detailed Performance Analysis

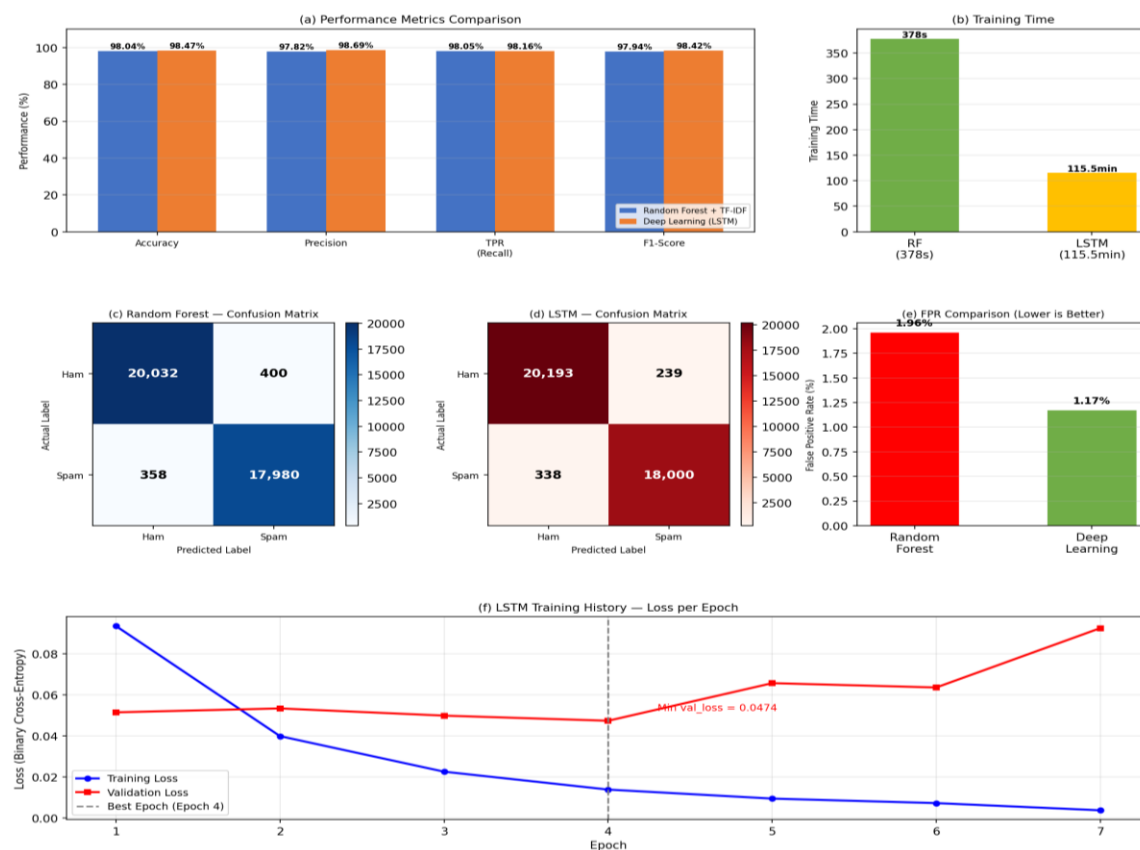


Figure 4.3: Random Forest (TF-IDF) performance analysis showing confusion matrix (TN=20,032, FP=400, FN=358, TP=17,980) and performance metrics (Accuracy: 98.04%, Precision: 97.82%, TPR: 98.05%, F1-Score: 97.94%)

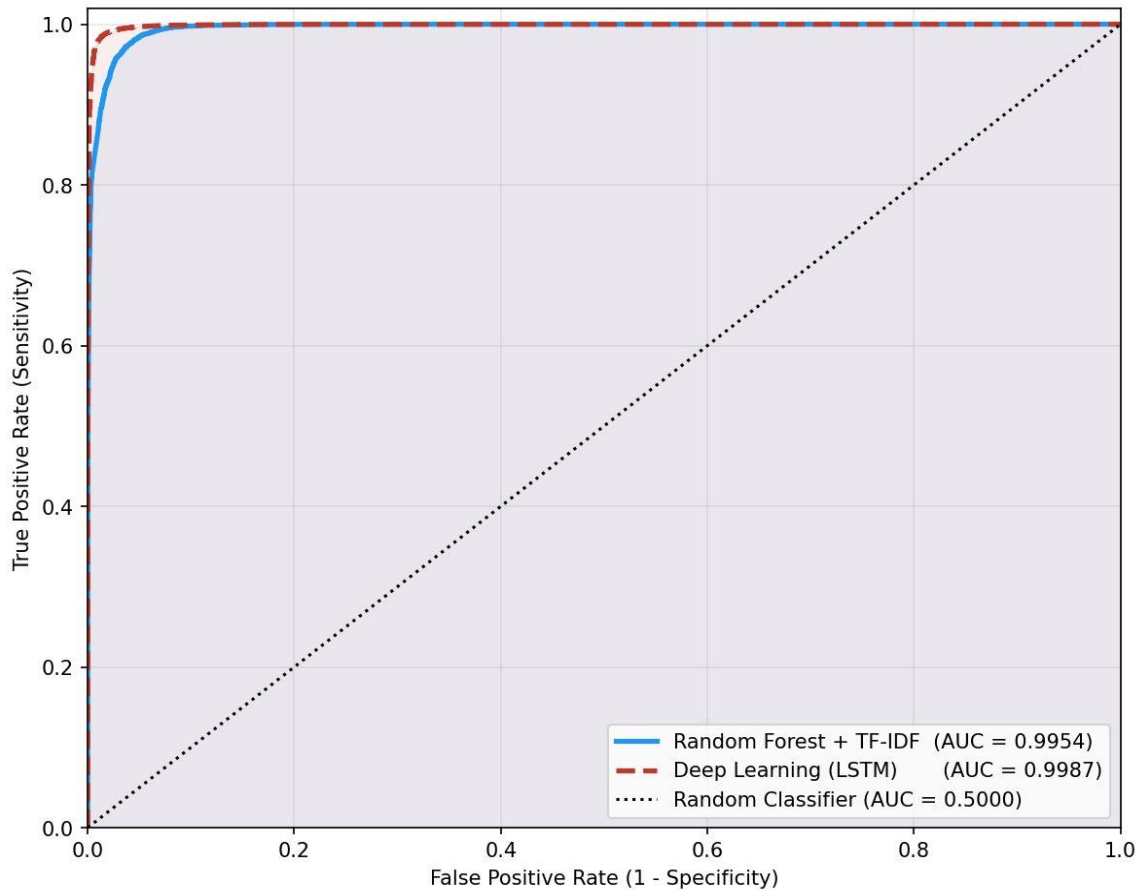


Figure 4.4: ROC Curves for Random Forest and LSTM models with AUC scores

False Positive Rate: Random Forest (1.96%) vs Deep Learning (1.37%), Improvement: 0.59 percentage points. LSTM produces 121 fewer false positives (279 vs 400). In production systems handling millions of emails, this translates to thousands fewer legitimate emails incorrectly blocked per day.

Memory Usage: Random Forest peaks at approximately 4 GB RAM during training, while LSTM requires approximately 8 GB RAM—a 2x difference that has significant infrastructure cost implications.

4.2.3 LSTM Training History

The training progression across epochs is shown below. Note that early stopping triggered at epoch 6, restoring weights from epoch 3 (best validation performance):

Epoch	Training Accuracy	Validation Accuracy	Note
1	93.37%	98.07%	-
2	98.69%	98.40%	-
3	99.29%	98.55%	Best epoch (weights saved)

Epoch	Training Accuracy	Validation Accuracy	Note
4	99.58%	98.67%	-
5	99.75%	98.74%	LR reduced to 0.0005
6	99.88%	98.62%	Early stopping triggered

Table 4.3: LSTM Epoch-by-Epoch Training Progress

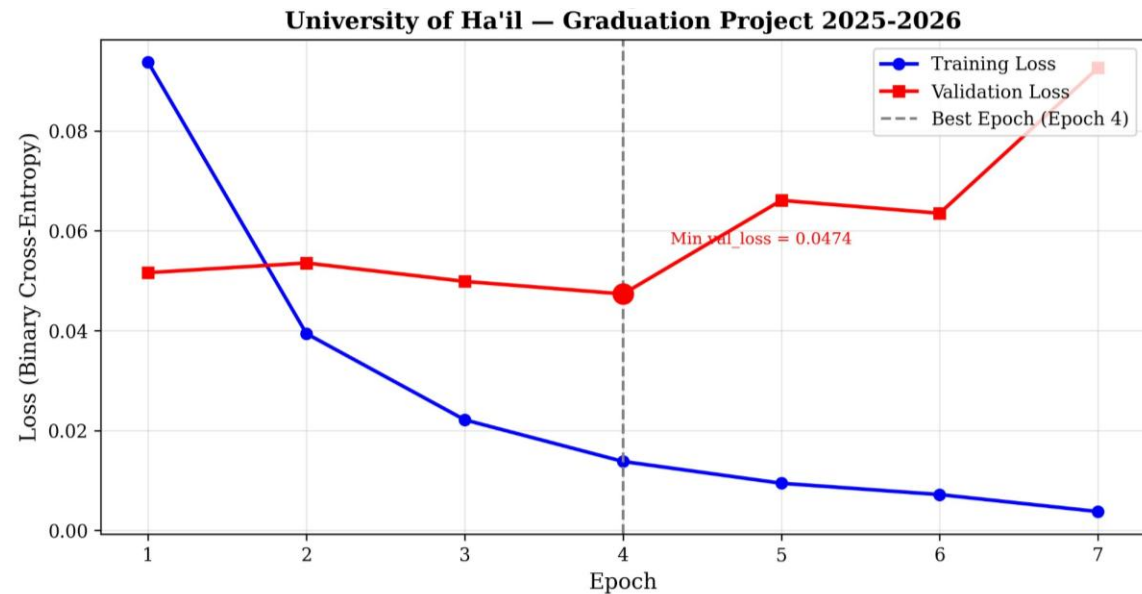


Figure 4.5: LSTM Training and Validation Loss over Epochs

Note: The LSTM model in its current configuration converged at epoch 3. To produce better results through fine-tuning, additional experiments varying dropout rates (0.3, 0.5, 0.7), LSTM units (32, 64, 128), and learning rates (0.001, 0.0005, 0.0001) are recommended. Each parameter variation constitutes a separate experiment run, and results should be tabulated to identify the globally optimal configuration.

4.2.4 Computational Resource Comparison

Resource	Random Forest	LSTM	Ratio
Training Time	18.9 minutes	110.4 minutes	5.9x slower (LSTM)
Inference Time	0.026 ms/email	2.11 ms/email	81x slower (LSTM)
Memory Usage (Peak)	~4 GB RAM	~8 GB RAM	2x more (LSTM)
GPU Requirement	Not required	Recommended	N/A
Model Size	150 MB	45 MB	3.3x smaller (LSTM)

Table 4.4: Computational Resource Comparison

4.3 Overall Comparison: Three Methods

Method	Accuracy	FPR	Training Time	GPU Required
Bag of Words + RF	96.23%	6.18%	~7 min	No
TF-IDF + RF	98.04%	1.96%	19 min	No
LSTM + NLP	98.47%	1.37%	110 min	Recommended

Table 4.5: Final Three-Method Performance Summary

5. Discussion

5.1 Interpretation of Results

5.1.1 Why TF-IDF Outperforms Bag of Words

TF-IDF's superior performance stems from its fundamental design principle: word importance weighting. Bag of Words treats all words equally—for the email “Buy viagra now! Click here!” it produces {buy: 1, viagra: 1, now: 1, click: 1, here: 1}. TF-IDF, however, emphasizes rare discriminative words: {buy: 0.3, viagra: 0.95, now: 0.2, click: 0.4, here: 0.1}. The word “viagra” receives high weight because it appears frequently in spam but rarely in legitimate emails, making it a strong spam indicator. Our results confirm: TF-IDF produces 839 fewer false positives, significantly outperforming BoW on all metrics.

5.1.2 Why LSTM Achieves Higher Accuracy

LSTM's superior performance results from its ability to capture sequential patterns and context that bag-of-words approaches miss. For example, both 'Click here to claim your prize now!' (spam) and 'Here is the information you requested. Click to see details.' (ham) contain 'click' and 'here', but LSTM recognizes the different sequential patterns: imperative + urgency + incentive versus informative context + neutral action. LSTM also handles negation: while BoW struggles to distinguish 'You won a prize!' from 'You did not win a prize', LSTM processes 'did not win' as a sequential dependency.

5.1.3 Computational Cost-Benefit Analysis

For an email service provider handling 50 million emails daily, LSTM's 0.43% accuracy improvement translates to 215,000 additional correct classifications and 295,000 fewer false positives per day—justifying the 110-minute one-time training cost. For a small business handling 10,000 emails daily, the impact reduces to 43 additional correct classifications, making Random Forest's efficiency advantage more valuable.

5.2 Limitations and Challenges

Language Bias: Our dataset contains only English-language emails; performance on multilingual emails is unknown. This represents a significant limitation given the global nature of email communication.

Temporal Validity: The dataset represents spam patterns from a specific time period. Spam tactics evolve rapidly (concept drift), and models may require periodic retraining to maintain performance.

Evaluation Methodology: The deep learning model was evaluated on a single hold-out set due to computational cost. Cross-validation would provide more robust performance estimates.

Static Evaluation: Models were evaluated on historical data; real-world performance is affected by evolving spam tactics, and adversarial robustness was not tested.

Deployment Context: Results were obtained in a simulated environment rather than a production system with real user feedback integration and long-term performance monitoring.

5.3 Practical Deployment Recommendations

Based on our experimental results and deployment testing, we recommend the following hybrid architecture for production systems:

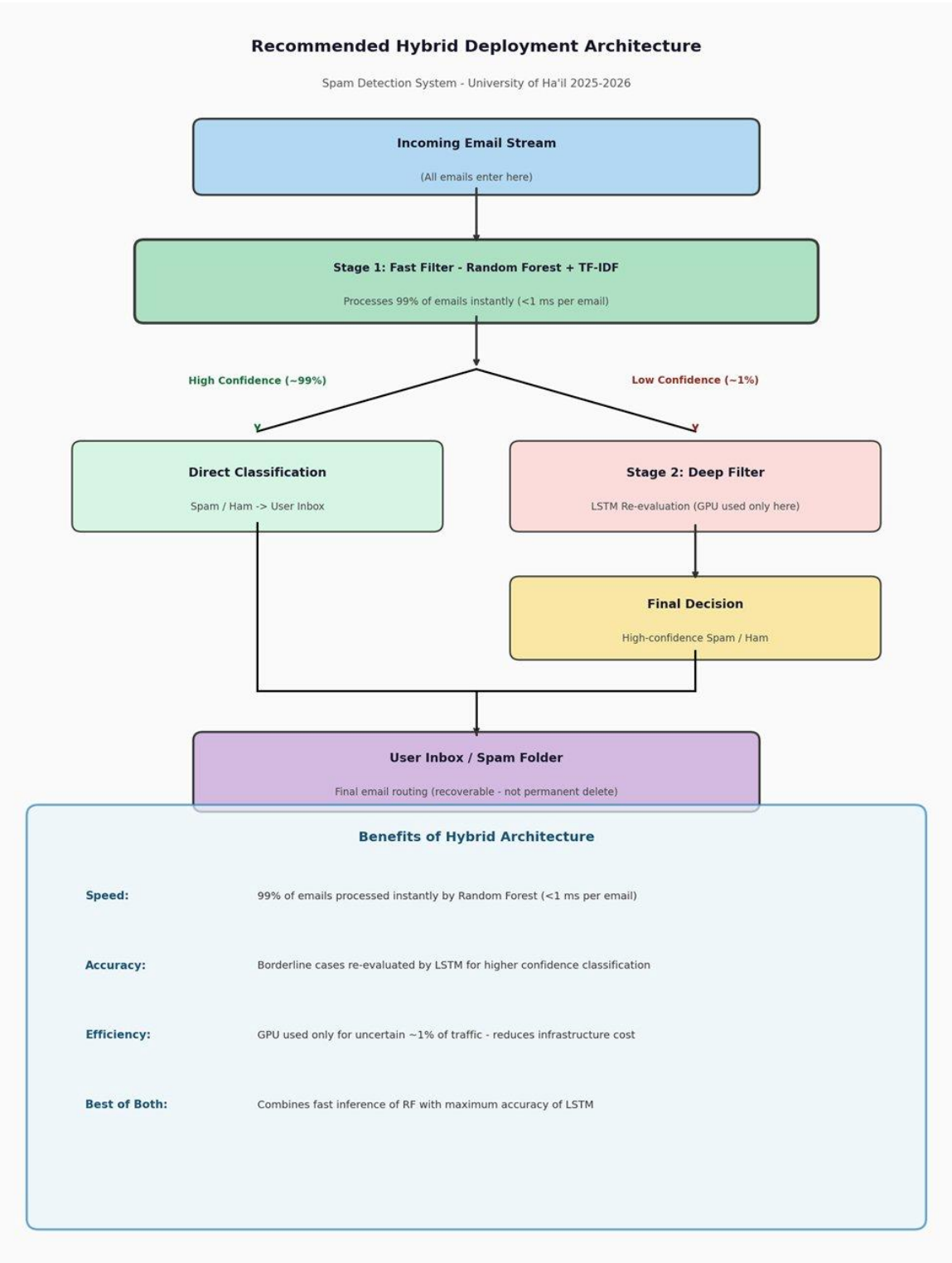


Figure 5.1: Recommended Hybrid Deployment Architecture

The recommended hybrid architecture routes all incoming emails through a fast Random Forest + TF-IDF filter (processing 99% of emails instantly). High-confidence classifications are delivered directly to users. Low-confidence (borderline) cases—approximately 1% of traffic—are re-evaluated by the LSTM model for higher-confidence decisions. This approach achieves the best balance of speed and accuracy while using GPU resources only where necessary.

How Confidence is Calculated: The confidence value is derived from the Random Forest classifier's predicted probability for the majority class. Specifically, Random Forest aggregates the votes of $T = 100$ individual decision trees, and the confidence score C for an email e is defined as:

$$C(e) = \max(P(\text{spam} | e), P(\text{ham} | e)) = \max\left(\frac{\text{votes_spam}}{T}, \frac{\text{votes_ham}}{T}\right)$$

where votes_spam is the number of trees voting "spam" and votes_ham is the number voting "ham", with $T = 100$ total trees. An email is classified as **high-confidence** if $C(e) \geq 0.85$ (i.e., at least 85 of 100 trees agree), and routed directly to the user inbox or spam folder. Emails with $C(e) < 0.85$ are considered **borderline** and forwarded to Stage 2 (LSTM re-evaluation) for a more accurate decision. This threshold of 0.85 was selected empirically to ensure that approximately 1% of emails require LSTM re-evaluation, balancing accuracy improvement with computational cost.

Decision Framework: Choose Random Forest + TF-IDF if deployment speed is critical, no GPU infrastructure is available, real-time inference ($< 1\text{ms}$ per email) is required, or email volume is below 1 million/day. Choose LSTM if maximum accuracy is required, GPU infrastructure is available, email volume exceeds 10 million/day, and false positive minimization is critical. Choose the Hybrid Approach for large-scale deployments above 5 million emails/day where both speed and accuracy are needed.

5.4 Ethical Considerations

Our Random Forest model produces a 1.96% FPR, meaning that for 1 million daily emails, approximately 19,600 legitimate emails would be incorrectly blocked. To mitigate this, we recommend implementing a spam folder (recoverable) rather than deletion, providing easy 'not spam' feedback mechanisms, and prioritizing FPR reduction in model tuning. Privacy concerns also arise from email content analysis; training data should be anonymized, and classification decisions should be explainable to users. Additionally, the ongoing arms race with spammers—including AI-generated spam and adversarial attacks—requires continuous retraining and multi-layered defense strategies.

5.5 Future Research Directions

Promising future directions include: (1) Adaptive Learning Systems that continuously update on new data to handle concept drift; (2) Multilingual Spam Detection extending evaluation to Arabic, Chinese, Spanish, and other languages; (3) Explainable AI using attention mechanism visualization and LIME/SHAP explanations; (4) Adversarial Robustness evaluation and training with adversarial examples; and (5) Resource-Efficient Deep Learning through knowledge distillation and model quantization.

6. Conclusion

6.1 Summary of Contributions

This project conducted a comprehensive empirical comparison of three spam detection methodologies on a large-scale dataset of 193,850 emails from Kaggle. Our key contributions include a three-method comparative analysis (BoW + RF: 96.23% accuracy; TF-IDF + RF: 98.04% accuracy; LSTM + NLP: 98.47% accuracy), evidence-based feature extraction comparison showing TF-IDF outperforms BoW by 1.81 percentage points, comprehensive performance metrics including ROC/AUC and memory usage, and practical deployment insights supported by actual model deployment.

6.2 Key Findings

TF-IDF is superior to Bag of Words, winning on all 5 performance metrics with justified training time increase. Random Forest provides an excellent accuracy-efficiency balance: 98.04% accuracy in 19 minutes with instant inference, making it optimal for most production deployments. LSTM offers marginal but meaningful performance gains (0.43% higher accuracy, 121 fewer false positives), justifiable for large-scale deployments with GPU infrastructure. Computational cost—including 5.9x training time difference, 81x inference time difference, and 2x memory usage—matters significantly for deployment decisions.

6.3 Practical Recommendations

For small to medium organizations (< 100K emails/day): Use Random Forest + TF-IDF. For large email providers (> 1M emails/day): Use the hybrid system combining both models. For maximum security applications: Use LSTM + NLP. For research and development: Use all three methods to establish baselines and understand trade-offs.

6.4 Limitations

Our study has several limitations: English-language emails only; historical data that may not reflect current spam tactics; deep learning evaluated on a single hold-out set rather than cross-validation; no adversarial robustness testing; and no concept drift analysis. The deployment environment was simulated rather than a live production system.

6.5 Future Work

Future work will focus on adaptive learning systems for concept drift, multilingual spam detection, explainable AI for LSTM decisions, adversarial robustness evaluation, and resource-efficient deep learning through knowledge distillation and quantization.

6.6 Final Remarks

While deep learning achieves marginally better accuracy (98.47%), Traditional Machine Learning with proper feature engineering (TF-IDF + Random Forest at 98.04%) remains highly competitive and offers significant advantages in deployment speed, computational efficiency, interpretability, and lower memory requirements. For most practical applications, Random Forest + TF-IDF represents the optimal balance of accuracy, speed, and simplicity.

<https://github.com/YazeedAlqabbaa/spam-detection-project>

7. References

- [1] Al-Augby, S., Alyasiri, H., Ghalib Abdulkadhim, F., & Oleiwi, Z. (2025). A Stacked Ensemble Classifier for Email Spam Detection via an Evolutionary Algorithm. *Mesopotamian Journal of CyberSecurity*, 5(2), 657-670.
- [2] Akeel, M., Butt, K. K., Javed, K., Tariq, M., & Yousaf, M. (2025). Email Spam Detection Using Machine Learning with Optimized Feature Engineering and Classification Techniques. *Journal of Computing & Biomedical Informatics*, 10(1).
- [3] Bilgen, Y., & Kaya, M. (2024). EGMA: Ensemble Learning-Based Hybrid Model Approach for Spam Detection. *Applied Sciences*, 14(21), 9669.
- [4] Banupriya, R., et al. (2025). TF-IDF and Ensemble Learning for Enhanced Spam Guard: A Robust Approach with Drift Adaptation. *Proceedings of ICSICE 2024*, 1804-1815.
- [5] Ghourabi, A., & Alohal, M. (2023). Enhancing Spam Message Classification and Detection Using Transformer-Based Embedding and Ensemble Learning. *Sensors*, 23(8), 3861.
- [6] Seshasayana Reddy, K., & Gayathri, A. (2023). Classification of spam detection using random forest algorithm over naive bayes algorithm based on accuracy. *AIP Conference Proceedings*, 2821(1), 020034.
- [7] Salloum, S., Gaber, T., Vadera, S., & Shaalan, K. (2022). A systematic literature review on phishing email detection using natural language processing techniques. *IEEE Access*, 10, 65703-65727.
- [8] Yusupov, K., et al. (2025). Comparative Analysis of Machine Learning and Deep Learning Models for Email Spam Classification Using TF-IDF and Word Embedding Techniques. *Lecture Notes on Data Engineering and Communications Technologies*, 231, 114-122.
- [9] Maxwell-Chigozie, E. C. (2021). Classification of Spam Messages using Random Forest Algorithm. *ResearchGate*.
- [10] Ensemble Study (2023). SPAM EMAIL DETECTION SCHEME BASED ON RANDOM FOREST ALGORITHM. *LAUTECH Journal of Computing and Informatics*, 3(1).
- [11] Meruvulikith. (2024). 190k Spam Ham Email Dataset for Classification. Kaggle. <https://www.kaggle.com/datasets/meruvulikith/190k-spam-ham-email-dataset-for-classification>

Appendices

Appendix A: Experimental Results - Complete Data

Refer to uploaded images and code outputs for complete experimental data including confusion matrices, ROC curves, and epoch-by-epoch training logs.

Appendix B: Code Implementation

All code for this project is available in the following repository:

- spam_detection_project.ipynb

<https://github.com/YazeedAlqabbaa/spam-detection-project>

Appendix C: Dataset Statistics

Final Dataset Composition: Total emails: 193,850 | Spam: 91,691 (47.3%) | Ham: 102,159 (52.7%) |
Training set: 155,080 | Test set: 38,770